
✓ Lab exercise PT-M1

✓ Step 1: Environment Setup and Dependency Installation

To construct a functional Retrieval-Augmented Generation (RAG) architecture, we must install the fundamental software stack handling data ingestion, vector representation, storage, and model inference:

- **langchain & langchain-community**:
 - *What it does*: Provides the core framework and integration modules to link various RAG components together into an executable chain.
 - *Why it is necessary for RAG*: RAG relies on a strict flow—querying a database, formatting retrieved context, and passing it to an LLM. LangChain orchestrates this entire lifecycle, bridging document retrievers with prompt templates and language models.
- **langchain-groq**:
 - *What it does*: Connects LangChain to Groq's high-performance API to run large language models (LLMs).
 - *Why it is necessary for RAG*: Serves as the **Generation** component of RAG. Once relevant documents are retrieved, the LLM processes both the user query and the retrieved context to generate a accurate, grounded response.
- **langchain-huggingface**:
 - *What it does*: Integrates HuggingFace embedding models directly into LangChain workflows.
 - *Why it is necessary for RAG*: Enables numerical feature extraction. Standard text cannot be mathematically searched; embedding models convert both raw document text and user queries into dense numerical vectors so their semantic similarity can be calculated.
- **chromadb**:
 - *What it does*: Acts as a specialized vector database for storing and querying text embeddings.

- *Why it is necessary for RAG:* Serves as the **Retrieval** backbone. It indexes embedded document chunks and performs fast approximate nearest-neighbor (ANN) searches to locate and retrieve the most relevant context chunks for any incoming user prompt.
- **pypdf** & **unstructured**:
 - *What it does:* Parses raw PDF files and unformatted document types into clean, plain-text strings.
 - *Why it is necessary for RAG:* Represents the **Data Ingestion** foundation. External knowledge bases are rarely stored as clean Python strings; these parsers extract the raw text from files so it can be split, embedded, and stored in the vector database.

```
!pip install -q langchain langchain-community langchain-groq langch
```

Step 2: Secure API Key Authentication and Environment Configuration

To utilize external LLM inference services within our RAG architecture without hardcoding sensitive credentials into the notebook source code, we configure environment-level authentication using Python's secure standard libraries:

- **import os**:
 - *What it does:* Imports Python's built-in operating system interface module, which provides access to environment variables.
 - *Why it is necessary for RAG:* LangChain integrations automatically look for environment variables (such as **GROQ_API_KEY**) when instantiating model wrappers. Accessing **os.environ** allows us to inject runtime credentials dynamically into the system environment without hardcoding them into the code base.
- **import getpass**:
 - *What it does:* Imports the standard Python library used to handle password and secret inputs securely without displaying characters on screen.
 - *Why it is necessary for RAG:* Protects sensitive credentials (like API keys) from being saved as plain text inside Jupyter Notebook execution histories, output logs, or version control commits.
- **if "GROQ_API_KEY" not in os.environ:**

- *What it does:* Checks if the `GROQ_API_KEY` environment variable has already been set in the active execution context.
 - *Why it is necessary for RAG:* Avoids repeatedly prompting the user for credentials every time the notebook cell is executed or re-run during development.
- `os.environ["GROQ_API_KEY"] = getpass.getpass("Enter your Groq API Key: ")`:
 - *What it does:* Prompts the user to enter their API key via a masked password field and assigns that value directly to the runtime environment variable map.
 - *Why it is necessary for RAG:* Generative components in RAG rely on secure communication with the underlying LLM provider. This step ensures that the generation layer can authenticate successfully with the remote LLM inference engine (Groq) to synthesize answers based on retrieved context.

```
import os
import getpass

# Prompts for API key securely without saving or echoing plain text
if "GROQ_API_KEY" not in os.environ:
    os.environ["GROQ_API_KEY"] = getpass.getpass("Enter your Groq A
```

Step 3: Document Ingestion and Semantic Text Splitting

To make external knowledge accessible to a RAG system, source documents must be loaded from local storage and divided into smaller, manageable text segments suitable for vector embedding:

- `from langchain_community.document_loaders import DirectoryLoader, TextLoader`:
 - *What it does:* Imports utility classes designed to scan file directories and parse plain-text files into LangChain `Document` objects containing page content and metadata.
 - *Why it is necessary for RAG:* RAG pipelines require structured access to external knowledge bases. These loaders automate reading multiple files from disk into memory so their contents can be processed

programmatically.

- `from langchain_text_splitters import RecursiveCharacterTextSplitter`:

- *What it does*: Imports a text splitter that recursively breaks documents down into smaller segments using a hierarchy of separator characters (like double newlines, single newlines, spaces, and individual characters).
- *Why it is necessary for RAG*: LLMs have finite context windows, and vector similarity search works far better on focused semantic concepts than on massive wall-of-text files. Splitting ensures retrieved context chunks fit cleanly within prompt templates without overwhelming the model.

- `os.makedirs("my_data", exist_ok=True)`:

- *What it does*: Ensures the destination folder (`my_data`) exists on the filesystem, creating it if it is missing without throwing an exception.
- *Why it is necessary for RAG*: Establishes a predictable directory structure for data ingestion, ensuring document loaders have a valid directory path to read from.

- `loader = DirectoryLoader(...)` & `loader_cls=TextLoader`:

- *What it does*: Configures a directory scanner that recursively traverses `my_data/` and parses matching text files using `TextLoader` while suppressing non-fatal read errors.
- *Why it is necessary for RAG*: Serves as the **Data Ingestion** layer. Using `TextLoader` explicitly ensures that raw text files (such as legal statutes) are parsed cleanly as UTF-8 strings without incurring dependencies on complex external parsing libraries.

- `raw_documents = loader.load()`:

- *What it does*: Executes the directory loading process and returns a list of raw `Document` objects containing file contents and associated metadata paths.
- *Why it is necessary for RAG*: Loads the raw knowledge corpus into memory so it can undergo chunking and vector index creation.

- `text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)`:

- *What it does*: Configures a chunking algorithm to divide text into maximum chunks of 500 characters. keeping a 50-character overlap

between adjacent chunks.

- *Why it is necessary for RAG:*
 - **chunk_size=500**: Keeps semantic chunks focused on specific legal provisions, ensuring high accuracy during vector retrieval.
 - **chunk_overlap=50**: Preserves contextual continuity across chunk boundaries, preventing key legal statements or definitions from being abruptly severed between two separate vectors.
- **documents = text_splitter.split_documents(raw_documents)**:
 - *What it does*: Takes the raw loaded documents and splits them into the finalized array of overlapped text chunks ready for vectorization.
 - *Why it is necessary for RAG*: Converts long-form static files into granular text units that can be converted into dense embeddings for the vector store.

```
import os
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Create target data directory
os.makedirs("my_data", exist_ok=True)

# Note: Ensure you put at least one file (.txt, .pdf, etc.) inside
# Uses TextLoader as fallback for plain text files to avoid unstructured
loader = DirectoryLoader(
    "my_data/",
    glob="**/*.*",
    show_progress=True,
    loader_cls=TextLoader,
    silent_errors=True
)

raw_documents = loader.load()

if not raw_documents:
    print("Warning: No documents found in 'my_data/'. Please add files.")
else:
    # Split documents into smaller semantic chunks
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=500,
        chunk_overlap=50
    )
    documents = text_splitter.split_documents(raw_documents)
    print(f"Loaded {len(raw_documents)} raw document(s) and split into {len(documents)} chunks")
```

100%|██████████| 2/2 [00:00<00:00, 2109.81it/s]Loaded 2 raw documents

Step 4: Vector Embeddings, ChromaDB Storage, and Retriever Configuration

This step converts raw text chunks into dense mathematical vector representations and initializes a vector search index to enable efficient semantic document retrieval:

- **`from langchain_huggingface import HuggingFaceEmbeddings`:**
 - *What it does:* Imports the module that provides access to open-source transformer-based embedding models hosted on Hugging Face.
 - *Why it is necessary for RAG:* Computers cannot directly measure the semantic similarity of text using raw strings. Embedding models map natural language sentences into multi-dimensional vectors (numerical lists) where words with similar meanings reside close together in vector space.
- **`from langchain_community.vectorstores import Chroma`:**
 - *What it does:* Imports the interface for ChromaDB, an open-source vector database designed for embedding storage and similarity search.
 - *Why it is necessary for RAG:* Serves as the primary knowledge store for the RAG system. It stores the generated vector embeddings alongside their text chunks and metadata, allowing rapid similarity queries against thousands of documents.
- **`embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")`:**
 - *What it does:* Loads the lightweight, high-performance `all-MiniLM-L6-v2` Sentence Transformer model locally.
 - *Why it is necessary for RAG:* Converts text into 384-dimensional dense vectors. It balances fast execution speeds with high retrieval quality, making it ideal for processing domain-specific documents without requiring expensive remote embedding API calls.
- **`vectorstore = Chroma.from_documents(documents=documents, embedding=embeddings)`:**
 - *What it does:* Passes the split text chunks through the embedding model to convert them into vectors and indexes them into ChromaDB in memory.
 - *Why it is necessary for RAG:* Constructs the searchable database index.

Every chunk of legal text is stored with its calculated embedding vector so that incoming user queries can be matched against them mathematically.

- **retriever = vectorstore.as_retriever(...):**
 - *What it does:* Converts the Chroma vector database into a LangChain **Retriever** interface equipped with Maximum Marginal Relevance (MMR) search parameters.
 - *Why it is necessary for RAG:* Connects the database to the RAG chain architecture:
 - **search_type="mmr"**: Maximize Marginal Relevance balances finding chunks that are strictly relevant to the query while avoiding fetching duplicate or highly redundant information.
 - **search_kwargs={"k": 6, "fetch_k": 10}**: Fetches the top 10 most relevant candidate chunks (**fetch_k=10**) from the vector database and selects the 6 most diverse and relevant chunks (**k=6**) to send to the language model context window.

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

# Initialize open-source embedding model
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

# Store embeddings into Chroma vector database
vectorstore = Chroma.from_documents(
    documents=documents,
    embedding=embeddings
)

# Set vectorstore as a retriever
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 6, "fetch_k": 10}
)
```

Loading weights: 100%

103/103 [00:00<00:00, 2326.17it/

Step 5: LLM Initialization and Guardrailed Prompt Template Construction

This step configures the generation engine of the RAG pipeline and establishes

strict prompt guardrails to prevent hallucination and restrict responses to the retrieved domain context:

- `from langchain_groq import ChatGroq`:
 - *What it does*: Imports the interface class for interacting with Chat-tuned LLMs hosted on Groq's high-speed inference platform.
 - *Why it is necessary for RAG*: Serves as the **Generation** module in RAG. Once relevant text chunks are retrieved, an LLM is needed to interpret the retrieved passage and generate natural language answers to the user's prompt.
- `from langchain_core.prompts import ChatPromptTemplate`:
 - *What it does*: Imports the template class used to construct structured chat prompts with predefined system instructions and dynamic variable placeholders.
 - *Why it is necessary for RAG*: RAG requires combining static instructions, dynamic user questions, and retrieved context chunks into a single formatted payload before sending it to the model.
- `llm = ChatGroq(model_name="...", temperature=0)`:
 - *What it does*: Initializes the Groq LLM client using a specified model identifier and sets the sampling temperature to 0.
 - *Why it is necessary for RAG*:
 - `temperature=0`: Disables model randomness and output variability. For domain-specific applications (such as legal or technical qa), deterministic outputs are required so the model states strict facts rather than creative extrapolations.
- `system_prompt = (...)`:
 - *What it does*: Defines a system-level instruction set that explicitly instructs the model to rely solely on the `{context}` variable and defines an exact fallback phrase ("I cannot answer based on the provided domain data.") if information is missing.
 - *Why it is necessary for RAG*: Acts as the primary **Guardrail against Hallucinations**. Out-of-the-box LLMs tend to draw on pre-trained internal knowledge. Explicit prompt engineering forces the model to act purely as a reader/summarizer of retrieved context rather than an unconstrained text generator.
- `prompt = ChatPromptTemplate.from_messages(...)`:
 - *What it does*: Assembles the prompt structure by mapping the system

instructions (containing the `{context}` placeholder) to the system role and the user's input (`{input}`) to the human role.

- *Why it is necessary for RAG:* Creates an executable template that LangChain chains can dynamically populate during runtime with retrieved document chunks and incoming user queries.

```
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate

# Change model_name to an active Groq endpoint
llm = ChatGroq(
    model_name="openai/gpt-oss-20b", # or "llama-3.3-70b-versatile"
    temperature=0
)

# Custom domain system prompt
system_prompt = (
    "You are a specialized AI assistant for the user's uploaded documents. "
    "Answer questions strictly using ONLY the provided context below. "
    "If the answer cannot be found in the context, reply: 'I cannot answer this question.' "
    "Context:\n{context}"
)

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}"),
])
```

Step 6: Assembling the Full Retrieval-Augmented Generation Chain

This step links the retriever, the prompt template, and the LLM into a unified, end-to-end execution pipeline using LangChain's modular chain factory functions:

- **`from langchain.chains import create_retrieval_chain`:**
 - *What it does:* Imports the standard factory function for constructing top-level RAG chains that handle both document retrieval and question answering.
 - *Why it is necessary for RAG:* Manages the execution lifecycle. When a user sends a query, this chain automatically queries the retriever, passes the retrieved documents to the downstream chain, and formats the final output dictionary containing both the generated answer and retrieved context.

- `from langchain.chains.combine_documents import create_stuff_documents_chain`:
 - *What it does*: Imports the document combination chain builder that takes a list of `Document` objects, formats them into a single string (the "stuffing" strategy), and passes them into the prompt template.
 - *Why it is necessary for RAG*: Bridges the retrieved text chunks with the LLM. It populates the `{context}` variable in the prompt template with the actual text of all retrieved documents so the LLM can analyze them.
- `combine_docs_chain = create_stuff_documents_chain(llm, prompt)`:
 - *What it does*: Instantiates the document-processing sub-chain by combining the initialized LLM (`llm`) with the guardrailed prompt template (`prompt`).
 - *Why it is necessary for RAG*: Encapsulates the **Generation** phase. It ensures that whenever document context is provided, it is consistently formatted into the system prompt structure before hitting the LLM.
- `rag_chain = create_retrieval_chain(retriever, combine_docs_chain)`:
 - *What it does*: Connects the vector retriever (`retriever`) to the document-stuffing chain (`combine_docs_chain`) to build the final runnable `rag_chain`.
 - *Why it is necessary for RAG*: Completes the **Retrieval-Augmented Generation** architecture. It automates the entire flow: `User Query -> Vector Search -> Context Retrieval -> Prompt Formatting -> LLM Generation -> Final Response`.

```
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain

# Combine prompt and LLM to process context
combine_docs_chain = create_stuff_documents_chain(llm, prompt)

# Assemble full retrieval-augmented generation chain
rag_chain = create_retrieval_chain(retriever, combine_docs_chain)
```

Step 7: Executing Test Case 1 – In-Domain Retrieval and Synthesis

This step tests the primary function of the RAG system: accurately retrieving relevant domain knowledge and synthesizing an in-context summary for a valid, in-domain query:

- `user_query = "Summarize the key information found in the documents.":`
 - *What it does:* Sets an open-ended in-domain test prompt requesting a summary of the uploaded knowledge base (`Republic Act No. 3019`) and (`Act No. 3815`).
 - *Why it is necessary for RAG:* Evaluates whether the vector retriever can pull relevant, representative chunks when given a broad, topically valid query.
- `response = rag_chain.invoke({"input": user_query}):`
 - *What it does:* Executes the end-to-end `rag_chain`. The retriever converts `user_query` into an embedding, fetches the top k matching chunks from ChromaDB, formats them into the system prompt, and passes the payload to Groq's LLM to generate a response dictionary.
 - *Why it is necessary for RAG:* Triggers the full RAG workflow to produce both a generated answer string (`response["answer"]`) and a list of retrieved document objects (`response["context"]`).
- `print(response["answer"]):`
 - *What it does:* Displays the natural language answer generated by the LLM.
 - *Why it is necessary for RAG:* Verifies that the model correctly used the retrieved context to formulate an accurate summary without introducing external facts.
- `for i, doc in enumerate(response["context"]): print(...):`
 - *What it does:* Loops over the retrieved `Document` objects stored in `response["context"]` and prints the `source` metadata field for each chunk.
 - *Why it is necessary for RAG:* Provides essential **Observability and Provenance Verification**. Printing source metadata proves that the LLM's response was grounded in specific, real chunks retrieved from the vector database.

```
# Test Case 1: In-Domain Query
user_query = "Summarize the key information found in the documents.
response = rag_chain.invoke({"input": user_query})
```

```

print("--- DOMAIN QUERY ANSWER ---")
print(response["answer"])

print("\n--- RETRIEVED SOURCE CHUNKS ---")
for i, doc in enumerate(response["context"]):
    print(f"Chunk {i+1} Source:", doc.metadata.get("source", "Unkn

```

```

--- DOMAIN QUERY ANSWER ---
**Key Points from the Provided Documents (Republic Act No. 3019 – Ar

| Section | Main Requirement / Prohibition | Practical Implication |
|-----|-----|-----|
| **Section 7** | Public officers must file a **Statement of Assets,
| **Section 8** | Dismissal of a public officer for **unexplained we
| **Section 4(a)** | **Prohibition on private individuals** from eng
| **Section 3k** (as referenced) | Penalties for **bribe givers** ar

**Overall Summary**
Republic Act No. 3019 mandates that public officers disclose their f

--- RETRIEVED SOURCE CHUNKS ---
Chunk 1 Source: my_data/Republic Act No. 3019.txt
Chunk 2 Source: my_data/Republic Act No. 3019.txt
Chunk 3 Source: my_data/Republic Act No. 3019.txt
Chunk 4 Source: my_data/Republic Act No. 3019.txt
Chunk 5 Source: my_data/Republic Act No. 3019.txt
Chunk 6 Source: my_data/Republic Act No. 3019.txt

```

Step 8: Executing Test Case 2 – Out-of-Domain Query and Fallback Guardrail Testing

This step evaluates the system's safety guardrails and fallback mechanisms when faced with an out-of-domain query completely unrelated to the legal document corpus:

- `ood_query = "What is the recipe for a classic Spaghetti Carbonara?"`:
 - *What it does*: Sets an intentionally irrelevant, out-of-domain query (a cooking recipe) that has no semantic relation to Philippine legal statutes.
 - *Why it is necessary for RAG*: Tests system robustness and hallucination prevention. A reliable domain-specific RAG chatbot must gracefully decline to answer topics outside its provided knowledge base rather than making up information or relying on pre-trained parametric knowledge.
- `ood_response = rag_chain.invoke({"input": ood_query})`:

- *What it does:* Passes the out-of-domain query through the full `rag_chain`. The retriever still fetches candidate chunks from the vector database (based on closest vector distance), but those chunks contain legal text, not recipes.
- *Why it is necessary for RAG:* Evaluates how the LLM handles context that does not contain the answer to the user's prompt.
- `print(ood_response["answer"]):`
 - *What it does:* Outputs the generated string response from the LLM.
 - *Why it is necessary for RAG:* Verifies that the strict system prompt guardrail ("I cannot answer based on the provided domain data.") was successfully triggered, confirming that the model refused to output a recipe from its internal training data.
- `for i, doc in enumerate(ood_response["context"]):`
`print(...):`
 - *What it does:* Iterates through `ood_response["context"]` and prints the file path metadata for each retrieved vector chunk.
 - *Why it is necessary for RAG:* Demonstrates vector store behavior under forced queries. It proves that while vector retrieval will always return nearest-neighbor chunks regardless of query relevance, the prompt guardrails effectively block the LLM from using irrelevant context to answer off-topic questions.

```
# Test Case 2: Out-of-Domain Query
ood_query = "What is the recipe for a classic Spaghetti Carbonara?"
ood_response = rag_chain.invoke({"input": ood_query})

print("--- OUT-OF-DOMAIN QUERY ANSWER ---")
print(ood_response["answer"])

print("\n--- RETRIEVED SOURCE CHUNKS ---")
for i, doc in enumerate(ood_response["context"]):
    print(f"Chunk {i+1} Source:", doc.metadata.get("source", "Unknc

--- OUT-OF-DOMAIN QUERY ANSWER ---
I cannot answer based on the provided domain data.

--- RETRIEVED SOURCE CHUNKS ---
Chunk 1 Source: my_data/Act No. 3815.txt
Chunk 2 Source: my_data/Act No. 3815.txt
Chunk 3 Source: my_data/Act No. 3815.txt
Chunk 4 Source: my_data/Act No. 3815.txt
Chunk 5 Source: my_data/Act No. 3815.txt
Chunk 6 Source: my_data/Act No. 3815.txt
```

Step 9: Executing Test Case 3 – Cross-Document Context Retrieval and Synthesis

This step tests the system's ability to perform multi-document retrieval and synthesize an answer that integrates information across multiple distinct source files (`Act No. 3815` and `Republic Act No. 3019`):

- `docs_3815 = vectorstore.similarity_search("...", k=3) & docs_3019 = vectorstore.similarity_search("...", k=3):`
 - *What it does:* Performs two targeted similarity searches against the ChromaDB vector store, fetching the top 3 most relevant chunks specifically for `Act No. 3815` and the top 3 chunks for `Republic Act No. 3019`.
 - *Why it is necessary for RAG:* Demonstrates explicit multi-document retrieval. Single-query vector searches can sometimes over-index on a single document if its language matches the prompt more closely. Performing targeted searches guarantees balanced, dual-document chunk extraction.
- `combined_docs = docs_3815 + docs_3019:`
 - *What it does:* Concatenates the two lists of retrieved chunks into a unified list of 6 `Document` objects.
 - *Why it is necessary for RAG:* Prepares a comprehensive context payload that contains representative text from both legal statutes to be passed into the document-processing chain.
- `multi_response = combine_docs_chain.invoke(...):`
 - *What it does:* Directly calls the document-stuffing chain (`combine_docs_chain`), passing both the tailored cross-document query (`input`) and the manually assembled context list (`context`).
 - *Why it is necessary for RAG:* Tests the LLM's cross-document synthesis capability—evaluating whether the model can parse information spanning multiple distinct files simultaneously and generate a cohesive, structured comparative response.
- `print(multi_response):`
 - *What it does:* Prints the final synthesized answer generated by the LLM.
 - *Why it is necessary for RAG:* Validates that the output accurately cites and combines rules, offenses, or definitions from both laws without confusing source attributions or hallucinating

combining source attributions or handcrafting.

- **for i, doc in enumerate(combined_docs): print(...):**
 - *What it does:* Iterates over `combined_docs` and prints the `source` metadata field for each of the 6 chunks.
 - *Why it is necessary for RAG:* Provides proof of dual-document context ingestion, verifying that Chunks 1–3 originated from `Act No. 3815.txt` and Chunks 4–6 originated from `Republic Act No. 3019.txt`.

```
# Test Case 3: Cross-Document Query
docs_3815 = vectorstore.similarity_search("Act No. 3815 Revised Penal Code")
docs_3019 = vectorstore.similarity_search("Republic Act No. 3019 Anti-Graft and Corrupt Practices Act")

combined_docs = docs_3815 + docs_3019

multi_response = combine_docs_chain.invoke({
    "input": "List any offenses, prohibitions, or legal rules mentioned in the provided documents.",
    "context": combined_docs
})

print("--- MULTI-DOCUMENT ANSWER ---")
print(multi_response)

print("\n--- RETRIEVED SOURCE CHUNKS ---")
for i, doc in enumerate(combined_docs):
    print(f"Chunk {i+1} Source:", doc.metadata.get("source", "Unknown"))
```

```
--- MULTI-DOCUMENT ANSWER ---
**Act No. 3815 (Revised Penal Code)**
- **Article 216 – Possession of Prohibited Interest by a Public Officer**

**Republic Act No. 3019 (Anti-Graft and Corrupt Practices Act)**
- **Section 2(a) – Definition of Terms** – a legal rule that defines the terms used in the Act.
- **Title: “Corrupt Practices of Public Officers”** – indicates the scope of the law.

--- RETRIEVED SOURCE CHUNKS ---
Chunk 1 Source: my_data/Act No. 3815.txt
Chunk 2 Source: my_data/Act No. 3815.txt
Chunk 3 Source: my_data/Act No. 3815.txt
Chunk 4 Source: my_data/Republic Act No. 3019.txt
Chunk 5 Source: my_data/Republic Act No. 3019.txt
Chunk 6 Source: my_data/Republic Act No. 3019.txt
```

Code Markdowns are researched through Gemini

